Module 1: Object Oriented Design

The Foundation of Modern Software Engineering

1.0 Introduction & Historical Context

Why are we here? Before the 1970s, programming was largely procedural. Think of it like a recipe: "Do step A, then step B, then step C." This worked for small scripts, but as software grew complex (like operating systems), this approach turned into "spaghetti code"—a tangled mess of dependencies where changing one line broke ten others.

Historical Pivot: Alan Kay and researchers at Xerox PARC introduced Smalltalk, the first true Object-Oriented Programming (OOP) language. Their insight was biological: don't build software like a machine; build it like a cell. Cells (objects) have their own internal state and communicate via chemical signals (messages). This shift from "functions acting on data" to "objects interacting with each other" is the core of this module.

1.1 OOP vs. Procedural Programming

This is the most common exam theory question. You need to understand the *philosophical* difference, not just the syntax.

Deep Dive Theory

- **Procedural Programming (POP):** Focuses on the **process**. Data is passive; functions are active. Data moves openly between functions.
  - *Analogy:* A buffet. The food (data) sits out in the open, and anyone (functions) can come and grab it. If someone sneezes on the mashed potatoes (global data corruption), everyone gets sick.
- **Object-Oriented Programming (OOP):** Focuses on the **data**. Data is active and protected. Functions are bundled with the data they manipulate.
  - *Analogy:* A restaurant with waiters. You (the user) cannot walk into the kitchen and grab a steak. You must send a request (message) to a waiter (object method). The waiter ensures the request is valid before fulfilling it.

Comparison Table (Cheat Sheet)

Feature	Procedural (e.g., C, Pascal)	Object-Oriented (e.g., Java, C++)
Primary Focus	Algorithms and functions.	Data and objects.
Data Security	Low. Data is often global and exposed.	High. Data is encapsulated (hidden).

Program Division	Divided into functions.	Divided into objects.
Approach	Top-Down (Main breaks into sub-functions).	Bottom-Up (Build components, then assemble).
Inheritance	Not supported.	Supported (Code reusability).
Overloading	generally not supported.	Supported (Polymorphism).

Worked Examples

Scenario: A bank account transfer.

Example 1: Procedural Approach (C-style pseudocode) *In POP, the logic is separate from the data.*

```
struct Account {
    int id;
    double balance;
};

void transfer(struct Account *from, struct Account *to, double amount) {
    if (from->balance >= amount) {
        from->balance -= amount;
        to->balance += amount;
    }
}
// Data is exposed. Any function could accidentally do: from->balance = -1000;
```

Example 2: OOP Approach (Java-style) *In OOP, the Account protects its own balance.*

```
class Account {
    private double balance; // Hidden data

    public void transfer(Account to, double amount) {
        if (amount > 0 && this.balance >= amount) {
            this.withdraw(amount);
            to.deposit(amount);
        }
    }
}
// Data is safe. External code cannot set balance directly.
```

1.2 Core Concepts: Object, Class, and Meta-Class

1. The Class (The Blueprint)

A class is a user-defined data type that acts as a template. It defines *attributes* (state) and *methods* (behavior). It consumes no memory at runtime until an object is created (except for static members).

2. The Object (The Realization)

An object is a runtime instance of a class. It occupies memory.

- **Formula:** `Object = State (Data) + Behavior (Methods) + Identity (Unique Address)`

3. The Meta-Class (The "Class of a Class")

Advanced Concept often tested in competitive exams. If an object is an instance of a class, what is the class an instance of? In pure OOP languages (like Smalltalk or Python), classes themselves are objects. The template that creates these class-objects is called a **Meta-Class**.

- **Why it matters:** It allows programs to modify their own structure at runtime (Reflection).

Worked Examples

Example 3: Blueprint Analogy

- **Class:** The architectural drawing of a house. You cannot live in the drawing.
- **Object:** The physical house built at 123 Maple Street.
- **Meta-Class:** The standard guidelines/rules for drawing architectural blueprints.

Example 4: Code Implementation

```
// Class
class Car {
    String color; // Attribute
    void drive() { ... } // Behavior
}

// Instantiation
Car myFerrari = new Car(); // 'myFerrari' is the Object
```

1.3 Relationships Among Objects & Classes

Understanding how components interact is crucial for "Object-Oriented Design" (OOD).

1. Association

A general binary relationship. "Uses-a" relationship.

- *Example:* A `Student` uses a `LibraryCard`. They are independent; if the student leaves, the card still exists.

2. Aggregation ("Has-a" - Weak)

A specialized form of association. It represents a "Whole-Part" relationship where the part can exist without the whole.

- *Example:* A `Department` has `Teachers`. If the Department closes, the Teachers still exist and can join another department.

3. Composition ("Has-a" - Strong)

Strong ownership. The part *cannot* exist without the whole.

- *Example:* A `House` has `Rooms`. If you destroy the House, the Rooms are destroyed too.

4. Instantiation

The relationship between a Class and an Object. The process of creating an instance.

Diagrammatic Summary (UML Style)

```
[Whole] <>----- [Part]  (Aggregation: Empty Diamond)
[Whole] <#>---- [Part]  (Composition: Filled Diamond)
[User]  ------- [Used]  (Association: Simple Line)
```

Worked Example: The University System

Scenario: Modeling a University.

- **Association:** `Professor` and `Textbook`. A professor uses a textbook, but they are not part of each other.
- **Aggregation:** `University` and `Student`. The university aggregates students. If the university shuts down, students don't cease to exist; they transfer.
- **Composition:** `University` and `Department`. A generic "Computer Science Dept" makes no sense without the University it belongs to.

1.4 Message Passing

In OOP, objects communicate by sending messages (calling methods). A message consists of:

1. **The Receiver:** The object handling the request.
2. **The Method Name:** What you want done.
3. **Arguments:** Data needed to do it.

Advantages:

- **Decoupling:** The sender doesn't need to know *how* the receiver works, just *what* messages it accepts.
- **Distributed Systems:** Message passing is the basis for Client-Server and Microservices architectures.

1.5 The Four Pillars (Properties of OOP)

1. Encapsulation (Data Hiding)

Wrapping data and code together and restricting access.

- *Goal:* Protect object integrity.
- *Real World:* A capsule pill. You swallow the capsule (interface), but the chemicals (data) are hidden inside to work safely.

2. Abstraction

Showing only necessary details and hiding complexity.

- *Goal:* Reduce cognitive load.
- *Real World:* A car accelerator. You press the pedal; you don't need to know about the fuel injection timing or airflow manifold.

3. Inheritance

Creating new classes from existing ones.

- *Goal:* Reusability and hierarchy.
- *Types:* Single, Multilevel, Hierarchical. (Note: Java/C# do not support Multiple Inheritance of classes to avoid the "Diamond Problem").

4. Polymorphism

"Many forms." The ability of a message to be processed in more than one form.

- *Compile-time:* Method Overloading.
- *Run-time:* Method Overriding.

1.6 Case Study: The Smart Home System

Apply what you learned.

Problem: Design a control system for a Smart Home. **Entities:** LightBulb, Thermostat, User, Controller.

Design:

1. **Class** `Device` **:** Abstract base class (Abstraction). Contains `turnOn()` and `turnOff()` .
2. **Classes** `LightBulb` **&** `Thermostat` **:** Inherit from `Device` (Inheritance).
3. **Polymorphism:** `Controller` has a list of `Device` objects. When the user hits "Master Off", the Controller loops through the list calling `turnOff()` . The Bulb dims, the Thermostat enters eco-mode. Same method name, different behaviors.
4. **Encapsulation:** The `Thermostat` has a private variable `temperature` . You cannot set it to 500°C directly; you must use `setTemp()` , which contains safety logic.

1.7 Exam Strategies & Practice Bank

Common Student Mistakes

1. **Confusing Class vs. Object:** *Tip: If you can count them (e.g., 5 apples), they are objects. "Apple" is the class.*
2. **Aggregation vs. Composition:** *Tip: Ask "Does the part die if the whole dies?" Yes = Composition.*
3. **Ignoring Metaclasses:** *Tip: Memorize "Class of a Class" for 2-mark definitions.*

Practice Questions

Part A: Multiple Choice (1 Mark each)

1. **Which relationship is described as "Part-of" where the part cannot exist independently?** a) Association b) Aggregation c) Composition d) Inheritance
   - *Solution: c) Composition*
2. **Message passing corresponds to which concept in procedural languages?** a) Variable declaration b) Function calling c) Looping d) Pointer arithmetic
   - *Solution: b) Function calling*
3. **Encapsulation allows for:** a) Global access to data b) Hiding implementation details c) Multiple inheritance d) None of the above
   - *Solution: b) Hiding implementation details*

Part B: Short Answer (5 Marks)

1. **Differentiate between Aggregation and Association with a diagrammatic example.**
   - *Hint:* Define both. Use the "Student-LibraryCard" (Association) vs. "Car-Engine" (Composition/Aggregation) example. Draw the UML lines (solid vs. diamond).
2. **Explain the concept of a Meta-class. Why is it not present in all OOP languages?**
   - *Hint:* Define it as an object describing a class. Mention that static languages (C++) handle class info at compile time, while dynamic languages (Smalltalk/Python) handle it at runtime.

Part C: Long Answer (10 Marks)

1. **"OOP is better than Procedural Programming." Justify this statement using the properties of OOP. Provide a coding example where Procedural Programming fails to provide data security.**
   - *Structure:*
     - Introduction: Define both briefly.
     - Body Paragraph 1: Discuss Encapsulation (Security).
     - Body Paragraph 2: Discuss Inheritance (Reusability vs. Copy-pasting code).
     - Body Paragraph 3: Discuss Maintenance (Spaghetti code in POP).
     - Code Snippet: Show a struct with public data vs. a class with private data (similar to Section 1.1).

- Conclusion: OOP is better for large, complex systems; POP is fine for small scripts.

1.8 Unit Summary Cheat Sheet

Term	Quick Definition	Key Keyword
Object	Runtime entity, instance of class.	"Real thing"
Class	Blueprint / Template.	"Definition"
Encapsulation	Hiding data.	"Security"
Polymorphism	Same interface, different action.	"Flexibility"
Inheritance	Acquiring properties of parent.	"Reusability"

Java Programming: Comprehensive Exam Handbook

Module 2: Class and Object Properties

Author: Prof. [Name] **Course:** Object-Oriented Programming with Java **Target Audience:** University Students preparing for final examinations.

Table of Contents

1. Java Fundamentals: The Architecture
2. Language Basics: Data, Logic, and Flow
3. The Object-Oriented Core
4. Memory Management & Lifecycle
5. Advanced Class Features
6. String Handling & Manipulation
7. Input/Output & System Interaction
8. Exam Strategy & Common Pitfalls
9. Practice Bank

1. Java Fundamentals: The Architecture

1.1 The "Why" of Java

Java was designed to solve a specific problem: portability in a heterogeneous network environment. Unlike C++, which compiles to architecture-specific machine code, Java decouples the source code from the hardware.

Key Advantages:

1. **Platform Independence (WORA):** Write Once, Run Anywhere. This is achieved via the JVM.
2. **Object-Oriented:** Everything (except primitive types) is an object. This enforces modularity.
3. **Robust Memory Management:** Automatic Garbage Collection prevents common C/C++ errors like memory leaks and dangling pointers.
4. **Multithreaded:** Native support for concurrent execution.

1.2 The Bytecode and JVM Ecosystem

This is the most critical architectural concept for the exam. You must understand the difference between the compiler and the interpreter.

- **Source Code (.java):** Human-readable text.

- **Compiler (`javac`):** Translates source code into **Bytecode (.class)**. Bytecode is a set of instructions for a virtual machine, not a physical CPU.
- **JVM (Java Virtual Machine):** An abstract computing machine. It reads bytecode and translates it into native machine code (using JIT - Just In Time compilation) for the specific operating system it is running on.

The Reality Check: Students often conflate the JDK, JRE, and JVM.

- **JVM:** The engine (runs code).
- **JRE:** The engine + libraries (JVM + `rt.jar`).
- **JDK:** The kit for developers (JRE + compiler `javac` + debuggers).

2. Language Basics: Data, Logic, and Flow

2.1 Data Types

Java is **strongly typed**. You must declare the type of every variable.

Type	Size	Range	Default
`byte`	8-bit	-128 to 127	0
`short`	16-bit	-32,768 to 32,767	0
`int`	32-bit	-2^31 to 2^31-1	0
`long`	64-bit	-2^63 to 2^63-1	0L
`float`	32-bit	IEEE 754	0.0f
`double`	64-bit	IEEE 754	0.0d
`char`	16-bit	\u0000 to \uFFFF (Unicode)	\u0000
`boolean`	1-bit*	true / false	false

Exam Trap: `float f = 3.14;` is a compile-time error. By default, decimal literals are `double` . You must explicitly cast or suffix: `float f = 3.14f;` .

2.2 Operators & Control Flow

Standard C-style operators apply. However, pay attention to **Short-Circuit Logical Operators** (`&&` , `||`).

Example: Short-Circuit vs Bitwise

```
int x = 10;
if (x > 5 || x++ > 10) { // Since x > 5 is true, the second part is NEVER evaluat
```

```
    System.out.println(x); // Output: 10
}
if (x > 5 | x++ > 10) { // Bitwise OR evaluates BOTH sides.
    System.out.println(x); // Output: 11
}
```

2.3 Arrays

Arrays in Java are **objects**. They are dynamically allocated on the heap.

Declaration vs Instantiation:

```
int[] arr;          // Declaration (arr is null)
arr = new int[5];   // Instantiation (memory allocated, filled with default 0)
```

Common Mistake: Trying to access `arr.length()` implies it's a method. It is not. For arrays, it is a **property**: `arr.length` . For Strings, it is a method: `str.length()` .

3. The Object-Oriented Core

3.1 Class and Object

- **Class:** A blueprint or template. It defines state (variables) and behavior (methods).
- **Object:** An instance of a class. It occupies memory in the Heap.

3.2 Constructors

A block of code similar to a method that is called when an instance of an object is created.

Rules:

1. Must have the **same name** as the class.
2. Must **not** have a return type (not even `void`).
3. If you write *no* constructor, the compiler injects a **default no-arg constructor**.
4. If you write *any* constructor, the compiler stops providing the default one.

Worked Example: Constructor Chaining

```
class Student {
    String name;
    int age;

    // No-arg constructor
    Student() {
        this("Unknown", 18); // Calls the parameterized constructor
    }

    // Parameterized constructor
    Student(String name, int age) {
```

```
        this.name = name;
        this.age = age;
    }
}
```

3.3 The `this` Keyword

`this` refers to the **current object instance**. **Usage:**

1. **Disambiguation:** `this.x = x;` (Instance variable vs Local variable).
2. **Constructor Chaining:** `this()` calls another constructor in the same class.
3. **Passing Instance:** `someMethod(this);` passes the current object to a method.

3.4 Access Specifiers

Controls visibility. This is crucial for **Encapsulation**.

1. `private` : Visible ONLY within the class.
2. **Default (package-private)**: Visible within the same package. (No keyword).
3. `protected` : Visible within package AND subclasses (even if in different packages).
4. `public` : Visible everywhere.

4. Memory Management & Lifecycle

4.1 Garbage Collection (GC)

In C++, you `malloc` and `free` . In Java, you `new` , and the JVM "takes out the trash." GC runs on the Heap memory. It identifies objects that are **unreachable** (no active references point to them) and reclaims memory.

Can you force GC? No. You can suggest it via `System.gc()` , but the JVM is not obligated to listen.

4.2 The `finalize()` Method (Deprecated but in Syllabus)

- **Purpose:** Called by the GC just before the object is destroyed. Intended for cleanup (closing files/sockets).
- **Reality:** It is unpredictable. Never rely on it for critical logic.

5. Advanced Class Features

5.1 Static: Class vs. Instance

The `static` keyword means the member belongs to the **class**, not a specific object.

- **Static Variable:** One copy shared by ALL instances.
- **Static Method:** Can be called without creating an object (`Math.sqrt()`).

- **Constraint:** A static method **cannot** access non-static (instance) members directly. Why? Because `static` context exists before any object is created.

5.2 Call by Value vs. Call by Reference

The Hard Truth: Java is **ALWAYS Call by Value**.

- **Primitives:** The value itself is copied. Changing the parameter does not affect the original variable.
- **Objects:** The **value of the reference** (memory address) is copied.
  - If you change the fields of the object via the copied reference, the original object **is** affected.
  - If you reassign the reference itself (e.g., `obj = new Class()`), the original reference remains unchanged.

Proof:

```
void change(int[] arr) {
    arr[0] = 99; // Modifies the actual object on heap
    arr = new int[5]; // Reassigns local copy. Original reference is untouched.
}
```

5.3 Method Overloading

Polymorphism at compile-time. **Rules:**

1. Same method name.
2. **Different parameter list** (type, number, or order).
3. Return type changes **do not** count as overloading.

5.4 Nested and Inner Classes

- **Inner Class:** Defined inside another class. Has access to the outer class's private members.
- **Static Nested Class:** Defined inside, but static. Cannot access outer instance members.

6. String Handling & Manipulation

6.1 Immutability

`String` objects in Java are **immutable**. Once created, they cannot be changed.

```
String s = "Java";
s.concat(" Rules");
System.out.println(s); // Output: "Java"
```

Why? Because `concat` returns a *new* String. `s` still points to "Java". Correct usage: `s = s.concat(" Rules");`

6.2 String Constant Pool

To save memory, Java stores string literals in a special pool.

```
String s1 = "Hello";
String s2 = "Hello"; // s1 and s2 point to the SAME object in memory.
String s3 = new String("Hello"); // Forces creation of a NEW object on heap.

s1 == s2; // true (Same reference)
s1 == s3; // false (Different references)
s1.equals(s3); // true (Same content)
```

6.3 Mutable Strings: `StringBuffer` and `StringBuilder`

If you modify a string frequently (e.g., in a loop), use `StringBuilder`.

- **StringBuilder:** Fast, not thread-safe.
- **StringBuffer:** Slower, synchronized (thread-safe).

7. Input/Output & System Interaction

7.1 Command Line Arguments

`public static void main(String[] args)` The `args` array stores strings passed during execution. `java MyApp arg1 arg2 -> args[0]` is "arg1".

7.2 Reading Input

1. **Scanner (** `java.util.Scanner` **):**
   - Parses primitive types and strings using regular expressions.
   - Slower but easier to use.
   - `Scanner sc = new Scanner(System.in); int x = sc.nextInt();`
2. **BufferedReader (** `java.io.BufferedReader` **):**
   - Reads text from a character-input stream.
   - Fast. Buffered for efficiency.
   - Returns only Strings (needs parsing).
   - `BufferedReader br = new BufferedReader(new InputStreamReader(System.in));`

8. Exam Strategy & Common Pitfalls

Common Student Mistakes (The "Fail" List)

1. `==` **vs** `.equals()` : Using `==` to compare Strings. You will lose marks instantly.

2. **Static Access:** Accessing static variables via an instance (`obj.staticVar`). It works but is bad practice. Use `ClassName.staticVar` .
3. **Array Index:** Forgetting that arrays are 0-indexed. `arr[arr.length]` throws `ArrayIndexOutOfBoundsException` .
4. **Void Constructors:** Writing `void MyClass() { ... }` . This is a *method*, not a constructor.
5. **Swallowing Exceptions:** In I/O code, writing `try { ... } catch (Exception e) {}` . Never leave a catch block empty. Print the stack trace.

Model Answer Structure

When asked to "Explain X":

1. **Definition:** One precise sentence.
2. **Syntax:** General code form.
3. **Example:** A 3-line code snippet.
4. **Key Point/Exception:** A "Did you know?" fact (e.g., String immutability).

9. Practice Bank

Section A: Multiple Choice Questions

Q1. What is the output of the following code?

```
public class Test {
    public static void main(String[] args) {
        String s1 = "Java";
        String s2 = new String("Java");
        System.out.println(s1 == s2);
        System.out.println(s1.equals(s2));
    }
}
```

A) true true B) false false C) false true D) true false **Answer:** C. `==` checks reference (address), `equals` checks content.

Q2. Which keyword is used to access the hidden member variable of a class? A) super B) this C) static D) void **Answer:** B. `this` refers to the current object instance.

Section B: Theory & Short Notes

Q3. Differentiate between `StringBuffer` and `String` .

- **String:** Immutable. Every modification creates a new object in memory. High memory overhead for heavy manipulation. Thread-safe (because it's immutable).
- **StringBuffer:** Mutable. Modifications happen in-place. Synchronized (thread-safe) methods, making it slower than StringBuilder but safe for multi-threading.

Q4. Explain "Write Once, Run Anywhere". Refers to Java's ability to compile source code into platform-neutral Bytecode (.class files). This bytecode is executed by the JVM, which translates it to the specific machine code of the host OS. Thus, the same compiled code runs on Windows, Linux, or Mac without recompilation.

Section C: Long Coding Questions

Q5. Design a class `Employee`.

- **Attributes:** `id` (int), `name` (String), `salary` (double), static `companyName` (String).
- **Methods:** Constructor to initialize fields, method to display details, static method to change company name.
- **Main:** Create 3 objects, change company name once, and display all.

Solution:

```
class Employee {
    int id;
    String name;
    double salary;
    static String companyName = "TechCorp"; // Shared by all

    // Constructor
    Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    void display() {
        System.out.println(id + " | " + name + " | " + salary + " | " + companyNc
    }

    static void changeCompany(String newName) {
        companyName = newName;
    }

    public static void main(String[] args) {
        Employee e1 = new Employee(101, "Alice", 50000);
        Employee e2 = new Employee(102, "Bob", 60000);

        System.out.println("Before Company Change:");
        e1.display();
        e2.display();

        Employee.changeCompany("GlobalTech");

        System.out.println("\nAfter Company Change:");
        e1.display(); // Both will show GlobalTech
        e2.display();
    }
}
```

Q6. Demonstrate Method Overloading with a `Calculator` class. Create methods `add` that can handle:

1. Two integers.
2. Three integers.
3. Two doubles.

Solution:

```
class Calculator {
    // Overload 1: Two ints
    int add(int a, int b) {
        return a + b;
    }

    // Overload 2: Three ints
    int add(int a, int b, int c) {
        return a + b + c;
    }

    // Overload 3: Two doubles
    double add(double a, double b) {
        return a + b;
    }

    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println("Int Add: " + calc.add(10, 20));
        System.out.println("3 Int Add: " + calc.add(10, 20, 30));
        System.out.println("Double Add: " + calc.add(10.5, 20.5));
    }
}
```

Module 3: Inheritance and Packages

Comprehensive Exam Handbook & Study Guide

Prof. [Your Name]

Contents

1 Introduction to Module 3 **2**

2 Inheritance: Theory and Application **2**
2.1 Definition and Purpose 2
2.2 Types of Inheritance 2
2.3 The `super` Keyword 2
2.4 The `final` Keyword 3

3 Polymorphism and Abstraction **3**
3.1 Dynamic Method Dispatch 3
3.2 Abstract Classes vs. Interfaces 4

4 Packages: Organizing Code **5**
4.1 Definition 5
4.2 Creation and Importing 5
4.3 Member Access Modifiers 5

5 Exam Strategies & Unit Summary **6**
5.1 Professor's "No-Nonsense" Exam Tips 6
5.2 Revision Cheat Sheet 6

6 Practice Questions **6**
6.1 Part A: Multiple Choice (2 Marks Each) 6
6.2 Part B: Short Answer (5 Marks) 7
6.3 Part C: Long Answer / Coding (10 Marks) 7

7 Case Study: The University Portal **8**

1 Introduction to Module 3

This module covers the backbone of Object-Oriented Programming (OOP): **Inheritance** and **Packages**. If you do not master these concepts, you cannot write scalable software. Period. In the exam, this module typically accounts for 20-25% of the total marks, focusing heavily on code analysis ("What is the output?") and architectural design ("Design a class hierarchy...").

The Professor's Warning

Brutal Truth: Most students fail questions in this module not because they don't know the syntax, but because they don't understand *memory allocation* during inheritance. If you think `super()` is optional or that `private` members are inherited, you will lose marks. Read this chapter thoroughly.

2 Inheritance: Theory and Application

2.1 Definition and Purpose

Inheritance is the mechanism by which one class acquires the properties (fields) and behaviors (methods) of another class. It represents an **IS-A** relationship (e.g., a `Car` IS-A `Vehicle`).

Why use it?

1. **Method Overriding:** To achieve runtime polymorphism.
2. **Code Reusability:** Write the code once in the parent class and reuse it in child classes.

2.2 Types of Inheritance

1. **Single Inheritance:** Class B extends Class A.
2. **Multilevel Inheritance:** Class C extends Class B, which extends Class A.
3. **Hierarchical Inheritance:** Class B and Class C both extend Class A.
4. **Multiple Inheritance (Not supported by Classes in Java):** Java does not support multiple inheritance with classes to avoid the "Diamond Problem" (ambiguity in method calls). It is achieved via Interfaces.

2.3 The `super` Keyword

The `super` keyword is a reference variable used to refer to the immediate parent class object.

Usage Contexts:

- **Variables:** To refer to the parent class instance variable if hidden by the child class (shadowing).
- **Methods:** To invoke the parent class method.
- **Constructors:** `super()` is used to invoke the parent class constructor.

Worked Example 1: Use of super

Problem: Demonstrate how `super` resolves ambiguity between parent and child class variables.

```java
class Animal {
    String color = "white";

    Animal() {
        System.out.println("Animal created");
    }
}

class Dog extends Animal {
    String color = "black";

    Dog() {
        super(); // Calls Animal constructor
        System.out.println("Dog created");
    }

    void printColor() {
        System.out.println(color);       // prints black (current class)
        System.out.println(super.color); // prints white (parent class)
    }
}

public class TestSuper {
    public static void main(String args[]) {
        Dog d = new Dog();
        d.printColor();
    }
}
```

2.4 The `final` Keyword

The `final` keyword is used to restrict the user. It can be applied in three contexts:

Context	Effect
Variable	The value becomes constant (cannot be changed).
Method	The method cannot be overridden by subclasses.
Class	The class cannot be inherited (subclassed).

3 Polymorphism and Abstraction

3.1 Dynamic Method Dispatch

Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at run-time, rather than compile-time. This is the foundation of runtime polymorphism.

The Golden Rule: A superclass reference variable can refer to a subclass object.

```
Parent p = new Child();
p.method(); // Calls Child's version of method()
```

Worked Example 2: Dynamic Method Dispatch

Scenario: A banking system where different banks give different interest rates.

```java
class Bank {
    float getRateOfInterest() { return 0; }
}

```

```java
class SBI extends Bank {
    float getRateOfInterest() { return 8.4f; }
}

class ICICI extends Bank {
    float getRateOfInterest() { return 7.3f; }
}

class AXIS extends Bank {
    float getRateOfInterest() { return 9.7f; }
}

public class TestPolymorphism {
    public static void main(String args[]) {
        Bank b; // Reference variable

        b = new SBI();
        System.out.println("SBI Rate: " + b.getRateOfInterest());

        b = new ICICI();
        System.out.println("ICICI Rate: " + b.getRateOfInterest());
    }
}
```

3.2 Abstract Classes vs. Interfaces

This is a guaranteed interview and exam question.

Abstract Class	Interface
Can have abstract and non-abstract methods.	Can have only abstract methods (until Java 8).
Uses the `extends` keyword.	Uses the `implements` keyword.
A class can extend only one abstract class.	A class can implement multiple interfaces.
Can have `final`, `non-final`, `static` variables.	Variables are implicitly `public static final`.

Worked Example 3: Interface Implementation

Problem: Design a system for 'Drawable' objects like Rectangle and Circle.

```java
interface Drawable {
    void draw();
}

class Rectangle implements Drawable {
    public void draw() {
        System.out.println("drawing rectangle");
    }
}

class Circle implements Drawable {
    public void draw() {
        System.out.println("drawing circle");
    }
}

class TestInterface {
```

```
    public static void main(String args[]) {
        Drawable d = new Circle();
        d.draw();
    }
}
```

4 Packages: Organizing Code

4.1 Definition

A package is a group of similar types of classes, interfaces, and sub-packages. It provides access protection and namespace management.

4.2 Creation and Importing

To create a package, use the `package` keyword as the first statement in the Java source file.

```
package mypack;
public class Simple { ... }
```

To use it: `import mypack.Simple;` or `import mypack.*;`

4.3 Member Access Modifiers

Understanding visibility is crucial for packages.

Modifier	Class	Package	Subclass	World
public	Y	Y	Y	Y
protected	Y	Y	Y	N
no modifier	Y	Y	N	N
private	Y	N	N	N

Table 2: Access Levels

5 Exam Strategies & Unit Summary

5.1 Professor's "No-Nonsense" Exam Tips

1. **Don't guess outputs:** Trace the code line by line. If you see `new Child()`, the Child constructor is called, but ONLY after the Parent constructor finishes.
2. **Constructors are NOT inherited:** You cannot override a constructor. If you write a class with arguments in the constructor and try to subclass it without calling `super(args)`, your code will not compile.
3. **Interface variables are constants:** If you try to change a variable defined in an interface (e.g., `interface A { int x=10; }`), you will get a compile-time error.
4. **Abstract Classes:** You cannot instantiate an abstract class. Writing `AbstractClass obj = new AbstractClass();` is an instant zero on that question.

5.2 Revision Cheat Sheet

- **IS-A Relationship:** Inheritance.
- **HAS-A Relationship:** Aggregation.
- **Diamond Problem:** Solved by Interfaces.
- **Upcasting:** Child object to Parent reference (Automatic).
- **Downcasting:** Parent reference to Child object (Explicit, risky).

6 Practice Questions

6.1 Part A: Multiple Choice (2 Marks Each)

Q1. Which of these keywords is used to refer to the member of base class from a subclass?

- A) upper
- B) super
- C) this
- D) None of the mentioned

Answer: B

Q2. What happens if you make a constructor final?

- A) Compile time error
- B) Run time error
- C) It works fine
- D) None of the above

Answer: A (Constructors cannot be final, static, or abstract).

6.2 Part B: Short Answer (5 Marks)

Q1. Differentiate between Method Overloading and Method Overriding with examples.
Hint: Focus on signature (same vs. different) and scope (same class vs. parent-child).

Q2. Explain the significance of the `protected` access modifier in the context of package inheritance.

6.3 Part C: Long Answer / Coding (10 Marks)

Q1. Design a class hierarchy for an **Employee Management System**.

- Create a base class `Employee` with `name`, `id`, and an abstract method `calculateSalary()`.
- Create two subclasses: `FullTimeEmployee` (fixed salary) and `PartTimeEmployee` (hourly rate).
- Implement the `calculateSalary()` method in both.
- Write a `Main` class to demonstrate polymorphism by creating an array of Employees.

7 Case Study: The University Portal

Context: A university needs a system to manage People. Everyone is a `Person`. A `Person` has a name and address. A `Student` is a `Person` with a GPA. A `Professor` is a `Person` with a Salary.

Task: Using inheritance, model this system to prevent code duplication (DRY Principle).

Solution Architecture:

1. **Base Class:** `Person` (Contains `name`, `address`, `getDetails()`).
2. **Subclass 1:** `Student` (Extends `Person`, adds `GPA`, overrides `getDetails()`).
3. **Subclass 2:** `Professor` (Extends `Person`, adds `salary`, overrides `getDetails()`).

```java
abstract class Person {
    protected String name;

    public Person(String name) { this.name = name; }
    public abstract void showRole();
}

class Student extends Person {
    public Student(String name) { super(name); }

    @Override
    public void showRole() {
        System.out.println(name + " is a Student.");
    }
}

class Professor extends Person {
    public Professor(String name) { super(name); }

    @Override
    public void showRole() {
        System.out.println(name + " is a Professor.");
    }
}
```

This structure allows the University to create a `List<Person>` that can hold both Students and Professors, simplifying database management and reporting.

Comprehensive Exam Preparation Handbook

Module 4: Exception Handling

Estimated Study Time: 6 Hours (Lecture) + 12 Hours (Self-Study) **Prerequisites:** Object-Oriented Programming (Classes, Objects, Inheritance)

1. Unit Overview & Professor's Note

"If you don't handle exceptions, your users will handle your failures."

Exception handling is not just about preventing crashes; it is about writing robust, professional-grade code that can recover gracefully from the unexpected. In this unit, we move beyond "happy path" programming (where everything works perfectly) into the reality of software development: files go missing, networks fail, and users input garbage data.

Learning Objectives:

1. Understand the internal mechanism of the Java Exception hierarchy.
2. Master the control flow of `try`, `catch`, `finally`, `throw`, and `throws`.
3. Design and implement custom exception strategies for business logic.
4. Distinguish between "recovering" and "crashing" correctly.

2. Deep Dive: Exception Handling Basics

2.1 What is an Exception?

An exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions. Unlike a compilation error (which stops code from building), an exception happens at **runtime**.

The Call Stack & Stack Unwinding: When an error occurs in a method, the method creates an object (the Exception Object) and hands it off to the runtime system. This is called **throwing an exception**.

1. The runtime system searches the call stack for a method that contains a block of code that can handle the exception.
2. It starts with the method in which the error occurred and proceeds through the list of methods that were called to get there (reverse order of invocation).
3. This process is called **stack unwinding**.

2.2 The Hierarchy (The `Throwable` Class)

Everything that can be "thrown" in Java inherits from `java.lang.Throwable`.

```
graph TD
    A[java.lang.Throwable] --> B[java.lang.Error]
    A[java.lang.Exception] --> C[RuntimeException]
    A[java.lang.Exception] --> D[Other Exceptions]
    B --> E[OutOfMemoryError]
    B --> F[StackOverflowError]
    C --> G[NullPointerException]
    C --> H[IndexOutOfBoundsException]
    D --> I[IOException]
    D --> J[SQLException]
```

2.3 Checked vs. Unchecked Exceptions

This is the most common theoretical exam question. You must memorize this distinction.

Feature	Checked Exceptions	Unchecked Exceptions	Errors
Parent Class	`java.lang.Exception` (excluding RuntimeException)	`java.lang.RuntimeException`	`java.lang.Error`
Compiler Behavior	**Forces** you to handle (`catch`) or declare (`throws`).	Compiler ignores them.	Compiler ignores them.
Philosophy	Recoverable conditions (e.g., File not found).	Programming logic errors (e.g., Null pointer).	Serious system failures (e.g., JVM runs out of RAM).
Action	**Must** be anticipated.	**Should** be fixed in code, not caught.	Generally cannot be recovered from.

3. Keywords and Control Flow

3.1 The "Guard" Block: `try-catch`

The `try` block encloses code that *might* throw an exception. The `catch` block handles it.

Syntax:

```
try {
    // Risky code
} catch (ExceptionType1 e) {
    // Handler for Type 1
} catch (ExceptionType2 e) {
    // Handler for Type 2
}
```

Professor's Tip: Order matters! Always catch specific exceptions *before* general ones. If you catch `Exception` first, the specific blocks below it become unreachable code (a compilation error).

3.2 The "Cleanup" Block: `finally`

The `finally` block **always** executes, regardless of whether an exception occurred or was caught. It is crucial for closing resources (files, database connections).

Exam Trick: Does `finally` execute if there is a `return` statement in the `try` block?
Answer: YES. The `finally` block executes *before* the control actually returns to the caller. The only time `finally` does not execute is if `System.exit(0)` is called or the JVM crashes.

3.3 `throw` vs `throws`

Confusing these is an immediate failure in a viva/interview.

- `throw` : Used *inside* a method to explicitly generate an exception.
  - *Usage:* `throw new ArithmeticException("Division by zero");`
- `throws` : Used in the *method signature* to declare that this method *might* cause an exception that it does not handle itself.
  - *Usage:* `public void readFile() throws IOException { ... }`

4. Worked Examples

Example 1: The "Fragile" Division (Unchecked Exception)

Scenario: A simple calculator that might divide by zero.

```
public class DivisionExample {
    public static int divide(int numerator, int denominator) {
        return numerator / denominator;
    }

    public static void main(String[] args) {
        try {
            int result = divide(10, 0); // Risky call
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            // Specific handling
            System.err.println("Error: Cannot divide by zero.");
            // e.printStackTrace(); // Good for debugging
        } finally {
            System.out.println("Calculation attempt finished.");
        }
    }
}
```

Output:

```
Error: Cannot divide by zero.
Calculation attempt finished.
```

Example 2: File Handling (Checked Exception & Throws)

Scenario: Reading a file that might not exist. We propagate the error using `throws`.

```java
import java.io.*;

public class FileReaderDemo {
    // We declare 'throws IOException' because we don't handle it here.
    // We pass the responsibility to the caller (main).
    public static void readFile(String path) throws IOException {
        FileReader fr = new FileReader(path);
        BufferedReader br = new BufferedReader(fr);
        System.out.println(br.readLine());
        br.close();
    }

    public static void main(String[] args) {
        try {
            readFile("non_existent_file.txt");
        } catch (FileNotFoundException e) {
            System.out.println("User Alert: The file was not found on disk.");
        } catch (IOException e) {
            System.out.println("User Alert: Disk read error.");
        }
    }
}
```

Example 3: Nested Try-Catch & Logic Flow

Scenario: Handling multiple layers of errors in a banking transaction.

```java
public class TransactionSystem {
    public static void main(String[] args) {
        try {
            // Outer Try: Connection
            System.out.println("1. Opening Database Connection...");

            try {
                // Inner Try: Transaction
                System.out.println("2. Processing Payment...");
                int funds = 100;
                int cost = 200;

                if (cost > funds) {
                    throw new IllegalArgumentException("Insufficient Funds");
                }
            } catch (NullPointerException npe) {
                System.out.println("Inner Catch: Data corrupted.");
            }
            // Note: IllegalArgumentException is NOT caught in inner, so it bubbl
```

```
        } catch (IllegalArgumentException e) {
            System.out.println("Outer Catch: Transaction Failed - " + e.getMessag
        } finally {
            System.out.println("Finally: Closing Database Connection.");
        }
    }
}
```

Output:

```
1. Opening Database Connection...
2. Processing Payment...
Outer Catch: Transaction Failed - Insufficient Funds
Finally: Closing Database Connection.
```

5. User Defined Exceptions

Sometimes, standard Java exceptions (`IOException` , `NullPointerException`) don't describe your business logic failures accurately. You need to create your own.

Recipe for Custom Exceptions:

1. Create a class.
2. Extend `Exception` (for Checked) or `RuntimeException` (for Unchecked).
3. Create a constructor that accepts a String message and passes it to `super(message)` .

Worked Example: The `InvalidAgeException`

```
// Step 1 & 2: Extend Exception
class InvalidAgeException extends Exception {
    // Step 3: Constructor
    public InvalidAgeException(String message) {
        super(message);
    }
}

class VoterRegistration {
    public void register(int age) throws InvalidAgeException {
        if (age < 18) {
            // Use the custom exception
            throw new InvalidAgeException("User is " + age + ". Must be 18+ to vc
        }
        System.out.println("User registered successfully.");
    }

    public static void main(String[] args) {
        VoterRegistration app = new VoterRegistration();
        try {
            app.register(16);
        } catch (InvalidAgeException e) {
```

```
            System.out.println("Registration Blocked: " + e.getMessage());
        }
    }
}
```

6. Exam Strategies & Common Pitfalls

Strategy 1: The "Empty Catch" Block (The Silent Killer)

Mistake:

```
catch (Exception e) { } // DO NOT DO THIS
```

Why: This "swallows" the exception. The program continues as if nothing happened, but the data might be corrupted. **Exam Answer:** Always print the stack trace (`e.printStackTrace()`) or log the error in your exam solutions unless explicitly told to ignore it.

Strategy 2: Return in Try/Catch/Finally

This is a favorite trick in multiple-choice questions.

```
int test() {
    try {
        return 1;
    } catch (Exception e) {
        return 2;
    } finally {
        return 3;
    }
}
```

Result: The method returns **3**. The `finally` block overrides the return values from try or catch.

Strategy 3: Predicting Output

When tracing code:

1. Mark where the exception occurs.
2. Immediately jump to the matching `catch` block.
3. Skip any code between the error and the catch.
4. Execute the `catch` .
5. Execute the `finally` .
6. Continue execution *after* the finally block (unless the exception was not caught).

7. Unit Summary & Cheat Sheet

Keyword	Function	Code Block
try	Defines a block of code to test for errors.	`try { ... }`
catch	Defines a block of code to execute if an error occurs.	`catch (Exception e) { ... }`
finally	Code that executes regardless of result.	`finally { ... }`
throw	Creates a custom error.	`throw new Exception();`
throws	Declares an exception in method signature.	`void method() throws Exception`

Best Practice Summary:

- Use **Checked Exceptions** for recoverable conditions (Network down, File missing).
- Use **Unchecked Exceptions** for programming errors (Null pointer, bad logic).
- Always clean up resources in `finally` .
- Never catch `Throwable` or `Error` directly unless writing system-level frameworks.

8. Practice Bank

Part A: Multiple Choice Questions (2 Marks Each)

Q1: Which of the following is NOT a checked exception? A) `IOException` B) `SQLException` C) `ArithmeticException` D) `FileNotFoundException` *Answer: C (It is a RuntimeException)*

Q2: If a `try` block throws an exception, and there is no matching `catch` block, what happens to the `finally` block? A) It is skipped. B) It executes, and then the program crashes (propagates exception). C) It executes, and the program continues normally. D) Compilation error. *Answer: B*

Part B: Short Answer Questions (5 Marks Each)

Q3: Explain the difference between `throw` and `throws` with a code example. *Hint: Focus on "action" vs "declaration". Use the File Reader example from Section 4.*

Q4: Why is it considered bad practice to `catch (Exception e)` at the very beginning of your catch blocks? *Hint: Discuss "unreachable code" for specific exceptions and the danger of hiding bugs.*

Part C: Long Answer / Coding (10 Marks Each)

Q5: Design a `BankAccount` class with a `withdraw(double amount)` method.

- Create a custom Checked Exception `InsufficientFundsException` .

- Create a custom Unchecked Exception `InvalidTransactionException` (for negative amounts).
- Implement the method to throw these exceptions appropriately.
- Write a `main` method to test scenarios: Success, Overdraft, and Negative Input.

Q6: Analyze the following code. Does it compile? If yes, what is the output? If no, why?

```
try {
    throw new IOException("Disk Error");
} catch (Exception e) {
    System.out.println("Caught General");
} catch (IOException e) {
    System.out.println("Caught IO");
}
```

*Solution: It does **not** compile. `IOException` is a subclass of `Exception`. Since `Exception` is caught first, the second catch block is "unreachable code".*

9. Case Study: The Mars Climate Orbiter

Topic: The cost of unhandled data errors.

In 1999, NASA lost a $125 million Mars orbiter.

- **The Cause:** One engineering team used Metric units (Newtons), while another used English units (Pound-force).
- **The Code perspective:** This was essentially an **Unchecked Exception** (Data mismatch)

Module 5: Threads & Applet

Syllabus Coverage: Basics of multithreading, main thread, thread life cycle, creation of threads, priorities, synchronization, inter-thread communication, deadlocks, suspending/resuming, Applet basics, life cycle, event models, listeners, buttons & text fields.

5.1 Introduction to Multithreading

5.1.1 The Concept

In traditional operating systems, a "process" is the unit of execution. However, processes are heavy; they require their own memory space and system resources. **Multithreading** introduces a lighter unit of execution called a **thread**.

A thread is a path of execution within a process. A single process can have multiple threads running concurrently, sharing the same memory space (heap) but maintaining their own stack and program counter.

Feature	Process	Thread
Memory	Independent memory space.	Shared memory space (Heap).
Overhead	High context-switching overhead.	Low context-switching overhead (Lightweight).
Dependency	Independent of other processes.	Dependent on the creating process.
Communication	IPC (Inter-Process Communication) is slow.	Inter-thread communication is fast (shared vars).

5.1.2 The Main Thread

When a Java program starts, one thread begins running immediately. This is called the **Main Thread**. It is spawned by the JVM to execute the `public static void main(String[] args)` method.

- It is the parent from which other child threads are spawned.
- It is usually the last thread to finish execution (unless `System.exit()` is called).

Example: Controlling the Main Thread

```
public class MainThreadDemo {
    public static void main(String[] args) {
        Thread t = Thread.currentThread();
        System.out.println("Current thread: " + t);

        t.setName("MyMainThread");
        try {
```

```
            for(int n = 5; n > 0; n--) {
                System.out.println(n);
                Thread.sleep(1000); // Pause for 1 second
            }
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted");
        }
    }
}
```

5.2 Thread Creation and Life Cycle

5.2.1 Creating Threads

Java provides two mechanisms to create a thread.

Approach 1: Implementing the `Runnable` Interface (Recommended) This is better because Java supports single inheritance only. If you extend `Thread`, you cannot extend any other class.

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread running via Runnable interface.");
    }
}
// Usage:
// Thread t = new Thread(new MyRunnable());
// t.start();
```

Approach 2: Extending the `Thread` Class

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running via Thread class extension.");
    }
}
// Usage:
// MyThread t = new MyThread();
// t.start();
```

5.2.2 The Thread Life Cycle

A thread goes through several states during its life. Understanding this is crucial for exam questions regarding state transitions.

1. **New (Born):** The thread is created using the `new` operator but `start()` has not been called.
2. **Runnable (Ready):** `start()` has been called. The thread is ready to run and waiting for CPU time from the scheduler.

3. **Running:** The CPU is actually executing the code inside `run()`.
4. **Blocked/Waiting/Sleeping:** The thread is alive but inactive. It might be waiting for I/O, a lock, or sleeping.
5. **Terminated (Dead):** The `run()` method has completed execution.

Diagrammatic View: `New -> start() -> Runnable` <-> (Scheduler) <-> `Running -> Terminated`

- From `Running`, a thread can move to `Blocked` (via `sleep`, `wait`, or I/O).
- From `Blocked`, it moves back to `Runnable` (not directly to `Running`).

5.2.3 Thread Priorities

Java threads have priorities ranging from 1 to 10. The thread scheduler uses these to decide when to switch contexts.

- `Thread.MIN_PRIORITY` (1)
- `Thread.NORM_PRIORITY` (5) - Default
- `Thread.MAX_PRIORITY` (10)

```
Thread t = new Thread();
t.setPriority(Thread.MAX_PRIORITY); // Set priority to 10
System.out.println(t.getPriority());
```

Exam Tip: Setting a priority does not *guarantee* execution order. It is a suggestion to the underlying OS scheduler.

5.3 Synchronization & Inter-thread Communication

5.3.1 The Need for Synchronization

When multiple threads access a shared resource (like a variable or file) simultaneously, data inconsistency can occur. This is known as a **Race Condition**.

Monitor Concept: Java uses the concept of a "monitor." Only one thread can own a monitor at a given time. When a thread enters a locked monitor, all other threads attempting to enter must wait.

The `synchronized` Keyword: Can be applied to methods or blocks.

```
class Counter {
    int count = 0;
    // Only one thread can execute this at a time
    public synchronized void increment() {
        count++;
    }
```

```
}
```

5.3.2 Inter-thread Communication

Threads often need to talk to each other. The classic example is the **Producer-Consumer** problem. We use three methods defined in `Object` (not `Thread`):

- `wait()` : Tells the calling thread to give up the monitor and go to sleep until some other thread enters the same monitor and calls `notify()` .
- `notify()` : Wakes up a single thread that called `wait()` on the same object.
- `notifyAll()` : Wakes up all threads calling `wait()` on the same object.

Worked Example: Producer-Consumer

```
class Q {
    int n;
    boolean valueSet = false;

    synchronized int get() {
        while (!valueSet) {
            try { wait(); } catch(InterruptedException e) {}
        }
        System.out.println("Got: " + n);
        valueSet = false;
        notify();
        return n;
    }

    synchronized void put(int n) {
        while (valueSet) {
            try { wait(); } catch(InterruptedException e) {}
        }
        this.n = n;
        valueSet = true;
        System.out.println("Put: " + n);
        notify();
    }
}
```

5.3.3 Deadlocks

A deadlock occurs when two threads have a circular dependency on a pair of synchronized objects.

- Thread A holds Lock 1 and waits for Lock 2.
- Thread B holds Lock 2 and waits for Lock 1.
- Result: Neither can proceed.

Prevention: Avoid circular wait conditions; acquire locks in a consistent order; use `tryLock` (in advanced concurrency API).

5.3.4 Suspending and Resuming

Warning: The methods `suspend()`, `resume()`, and `stop()` are **deprecated** in modern Java because they are deadlock-prone.

- **Old way:** `t.suspend()` pauses, `t.resume()` restarts.
- **Modern way:** Use a `volatile boolean` flag in the `run()` loop to check if the thread should pause (wait) or run.

5.4 Applet Programming

Professor's Note: Applets are obsolete and were removed in recent Java versions (Java 11+). However, for the purpose of this syllabus and exam, we must study them as part of the AWT (Abstract Window Toolkit) history.

5.4.1 Basics and Architecture

An Applet is a small Java program meant to run within a web browser or Applet Viewer. It is a subclass of `java.awt.Panel`.

- **No Main Method:** Applets do not use `main()`. Execution begins when the browser loads the applet.
- **Sandboxing:** Applets run with restricted permissions (cannot read/write local files, run local executables) for security.

5.4.2 Applet Life Cycle

1. `init()`: Called once when the applet is first loaded. Used for initialization (variables, GUI components).
2. `start()`: Called after `init()`, and every time the browser tab is revisited. Starts animations or threads.
3. `paint(Graphics g)`: Called whenever the output needs to be redrawn (e.g., window resized/uncovered).
4. `stop()`: Called when the user navigates away from the page. Suspends threads.
5. `destroy()`: Called once when the browser is closed. Cleanup.

5.4.3 HTML Integration

The `<applet>` tag (also deprecated) is used to embed the code.

```
<applet code="MyApplet.class" width="300" height="300"> </applet>
```

5.5 Event Handling & Controls

5.5.1 Delegation Event Model

Modern Java AWT/Swing uses the Delegation Event Model.

1. **Source:** The GUI component generating the event (e.g., a Button).
2. **Listener:** An interface that receives the event (e.g., `ActionListener`).
3. **Event Object:** Encapsulates information about the event (e.g., `ActionEvent`).

Mechanism: The Source registers a Listener. When an event occurs, the Source calls a method on the Listener.

5.5.2 Creating Buttons and TextFields

- **Button:** `Button b = new Button("Submit");`
- **TextField:** `TextField tf = new TextField(20);` (Single line input).
- **Label:** `Label l = new Label("Name:");`

Worked Example: A Simple Counter Applet

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;

public class CounterApplet extends Applet implements ActionListener {
    TextField tfCount;
    Button btnCount;
    int count = 0;

    public void init() {
        tfCount = new TextField("0", 10);
        btnCount = new Button("Increment");

        // Register the listener (this class listens to the button)
        btnCount.addActionListener(this);

        add(new Label("Counter:"));
        add(tfCount);
        add(btnCount);
    }

    // Listener method
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == btnCount) {
            count++;
            tfCount.setText(String.valueOf(count));
        }
    }
}
```

5.6 Exam Preparation Section

5.6.1 Summary Cheat Sheet

Concept	Key Keyword/Class	Description

Inheritance	`extends Thread`	Easy, but limits class design (no multiple inheritance).
Interface	`implements Runnable`	Flexible, allows extending other classes. Preferred.
Sync	`synchronized`	Prevents race conditions.
Wait/Notify	`Object` methods	Used for inter-thread communication. Must be in sync block.
Init	`init()`	Applet setup. Called once.
Paint	`paint(Graphics g)`	Drawing logic. Never call directly; use `repaint()`.

5.6.2 Common Mistakes & Exam Strategies

1. **Calling `run()` instead of `start()`:**
   - *Mistake:* Calling `t.run()` executes the code in the *current* thread (like a normal method call), not a new thread.
   - *Correction:* Always call `t.start()` to create the new stack and context.
2. **Deadlock Definition:**
   - *Tip:* Don't just say "threads stop." Define it as "circular dependency on resources."
3. **Deprecation:**
   - *Strategy:* If asked to write code for suspending threads, add a comment: *"Note: `suspend()` is deprecated. Using a flag-based approach is best practice, but `suspend()` is shown here per syllabus requirements."* This impresses examiners.

5.6.3 Practice Questions

Part A: Multiple Choice

1. Which method is called to begin the execution of a thread? a) `init()` b) `start()` c) `run()` d) `resume()` *Answer: b) `start()`*
2. Which interface must be implemented to handle button clicks? a) `ItemListener` b) `WindowListener` c) `ActionListener` d) `MouseListener` *Answer: c) `ActionListener`*

Part B: Short Answer

1. **Differentiate between `yield()` and `sleep()`.** *Answer:* `sleep(t)` forces the thread to pause for time `t` and enter the Blocked state. `yield()` is a hint to the scheduler that the current thread is willing to give up usage of the CPU; it goes back to the Runnable state, not Blocked, and might run again immediately if no other threads of equal/higher priority are ready.
2. **Why are `wait()`, `notify()`, and `notifyAll()` defined in `Object` and not `Thread`?** *Answer:* These methods relate to locks (monitors). In Java, every Object has a lock, not just

Threads. Threads wait on an *object's* monitor, so the methods must be part of the `Object` class.

Part C: Long Answer / Coding

1. **Write a Java program to implement a Producer-Consumer problem using wait() and notify().** *(See section 5.3.2 for the solution logic. Ensure you include the `synchronized` keyword).*
2. **Explain the Life Cycle of a Thread with a neat diagram.** *(Draw the state transition diagram: New -> Runnable <-> Running -> Dead, with Blocked side-states. explain each*